

EDS Lab Assignments

Practical 4

Name: Ram Mahadev Pandhare

PRN: 202501040071

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

The usershould enter a list of numbers separated by space when prompted.

Output Format:

Theprogram should display the mean of even and odd numbers separately.
Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

1

import pandas as pd

2

3

Take inputs from the user to create a list of numbers

numbers = list(map(int, input().split()))

4

5

6

Create a Pandas series from the list of numbers

series = pd.Series(numbers)

7

8

Grouping by even and odd numbers and calculating the mean

grouped = series.groupby(series%2==0).mean()

9

10

11

Display the mean of even and odd numbers with labels

grouped.index = ["Even" if is_even else "Odd" for is_even in

12

grouped.index]

13

print("Mean of even and odd numbers:")

Average time

0.010 s

10.17 ms

Maximum time

0.021 s

21.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd ... 5.0

Even ... 6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd ... 5.0

Even ... 6.0

dtype: float64

Terminal

Test cases

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).

- Add the new row to the DataFrame.

- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.

- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.

- Remove the specified row.

- Display the DataFrame after deleting the row

Add a new column:

- Add a column `Gender` with values taken from the user.

- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.

- Display the DataFrame after modifying the column.

Delete a column:

- Remove the `Age` column.

- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")

```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = ['Age'])
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Output:

The screenshot displays a Jupyter Notebook environment. The left pane shows the notebook content, and the right pane shows the test results.

Notebook Content:

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Test Results:

Average time: 0.065 s, Maximum time: 0.065 s, 1 out of 1 shown test case(s) passed, 1 out of 1 hidden test case(s) passed.

Test case 1:

Expected output	Actual output
Original DataFrame:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35	Original DataFrame:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35
New name: Susan New age: 29	New name: Susan New age: 29
After adding a row:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29	After adding a row:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29
Index of row to modify: 2 New age: 27	Index of row to modify: 2 New age: 27
After modifying a row:Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29	After modifying a row:Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29
Index of row to delete: 3	Index of row to delete: 3

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

Display the first five rows of the data frame.

Calculate the average age of the students (limit the average age up to 2 decimal places).

Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# write your code here..
```

```
print("First five rows:")
```

```
print(data.head())
```

```
average_age = round(data["Age"].mean(), 2)
```

```
print(f"Average age: {average_age}")
```

```
filtered_students = data[data["Grade"] <= "B"]
```

```
print("Students with a grade up to B")
```

```
print(filtered_students)
```

Output:

The screenshot shows a code editor with the following content:

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

student...

Average time: 0.029 s
Maximum time: 0.040 s
29.00 ms
40.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1

Expected output

```
First five rows:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
3  Mike  23    C
4  Sony  21    B
Average age: 23.29
Students with a grade up to B:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
3  Sony  21    B
4  Aelia  26    A
```

Actual output

```
First five rows:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
3  Mike  23    C
4  Sony  21    B
Average age: 23.29
Students with a grade up to B:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
3  Sony  21    B
4  Aelia  26    A
```

Terminal

Test cases

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

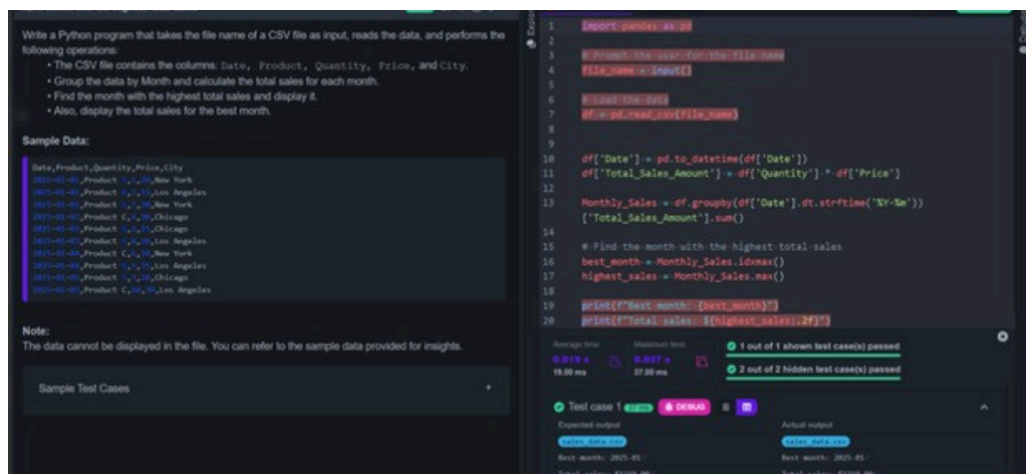
- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:



The screenshot shows a code editor with a Python program on the left and its output on the right. The program prompts for a file name, reads a CSV file, calculates monthly total sales, and identifies the month with the highest sales. The output shows the best month as 2023-01 and the total sales as \$1238.00.

```
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:
Date,Product,Quantity,Price,City
2023-01-01,Product A,10,12,New York
2023-01-01,Product B,5,8,Los Angeles
2023-01-01,Product C,3,5,New York
2023-01-01,Product C,2,5,Chicago
2023-01-01,Product A,10,12,Los Angeles
2023-01-01,Product A,10,12,New York
2023-01-01,Product A,10,12,Los Angeles
2023-01-01,Product C,10,12,Chicago
2023-01-01,Product C,10,12,Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases
```

```
1 import pandas as pd
2 # Prompt the user for the file name
3 file_name = input()
4 # Load the data
5 df = pd.read_csv(file_name)
6
7 df['Date'] = pd.to_datetime(df['Date'])
8 df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
9
10 Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))
11 ['Total_Sales_Amount'].sum()
12
13 # Find the month with the highest total sales
14 best_month = Monthly_Sales.idxmax()
15 highest_sales = Monthly_Sales.max()
16
17 print(f"Best month: {best_month}")
18 print(f"Total sales: ${highest_sales:.2f}")
```

Average time: 0.019 s | Memory: 0.007 s | 1 out of 1 shown test case(s) passed
18.00 ms | 37.00 ms | 2 out of 2 hidden test case(s) passed

Test case 1: ✓ ✗ ?

Expected output	Actual output
Best month: 2023-01	Best month: 2023-01
Total sales: \$1238.00	Total sales: \$1238.00

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Find the product that sold the most in terms of quantity sold.
Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

Output:

The screenshot displays a code editor with a Python program and its execution results. The code prompts the user for a file name, loads the data from a CSV file, groups the data by product to find the total quantity sold for each, identifies the product with the highest total quantity sold, and prints the results.

Sample Data:

Date	Product	Quantity	Price	City
2021-01-01	Product A	10	10	New York
2021-01-01	Product A	5	10	Los Angeles
2021-01-01	Product A	5	10	New York
2021-01-01	Product C	10	10	Chicago
2021-01-01	Product C	5	10	Chicago
2021-01-01	Product C	5	10	Los Angeles
2021-01-01	Product C	5	10	New York
2021-01-01	Product C	5	10	Los Angeles
2021-01-01	Product C	5	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case	Expected Output	Actual Output
Test case 1	Best selling product: Product A Total quantity sold: 20	Best selling product: Product A Total quantity sold: 20

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# write the code..
```

```
city_sales = df.groupby("City")["Quantity"].sum()  
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

Output:

The screenshot displays a code editor with the following content:

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2020-01-01	Product A	10	100	New York
2020-01-01	Product B	20	200	Los Angeles
2020-01-01	Product C	30	300	New York
2020-01-01	Product D	40	400	Chicago
2020-01-01	Product E	50	500	Chicago
2020-01-01	Product F	60	600	Los Angeles
2020-01-01	Product G	70	700	New York
2020-01-01	Product H	80	800	Los Angeles
2020-01-01	Product I	90	900	Chicago
2020-01-01	Product J	100	1000	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Results:

- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1:

Expected output	Actual output
City sold the most products: Los Angeles	City sold the most products: Los Angeles

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd
```

```
from itertools import combinations
```

```
from collections import Counter
```

```
# Prompt user to input the file name
```

```
file_name = input()
```

```
# Read data from the specified CSV file
```

```
df = pd.read_csv(file_name)
```

```
# write the code
```

```
grouped = df.groupby("Date")["Product"].apply(list)
```

```
pairs_counter = Counter()
```

```
for products in grouped:
```

```
    pairs = combinations(sorted(products), 2)
```

```
    pairs_counter.update(pairs)
```

```
max_frequency = max(pairs_counter.values())
```

```
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==  
max_frequency]
```

```
# Output the most frequent product pairs
```

```
for (product1, product2), frequency in most_common_pairs:
```

```
    print(f"{product1} and {product2}: {frequency} times")
```

Output:

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	1	10	New York
2025-01-01	Product B	1	10	Los Angeles
2025-01-01	Product C	1	10	New York
2025-01-01	Product C	1	10	Chicago
2025-01-01	Product A	1	10	Chicago
2025-01-01	Product A	1	10	Los Angeles
2025-01-01	Product C	1	10	New York
2025-01-01	Product A	1	10	Los Angeles
2025-01-01	Product C	1	10	Chicago
2025-01-01	Product C	1	10	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)

frequency...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 grouped = df.groupby("Date")["Product"].apply(list)
13
14 pairs_counter = Counter()
15 for products in grouped:
16     pairs = combinations(sorted(products), 2)
17     pairs_counter.update(pairs)
18
```

Average time

0.019 s

Maximum time

0.028 s

15.00 ms

20.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

DEBUG

Actual output

Expected output

Product A and Product B: 2 times

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product C: 2 times

Terminal

Test cases

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())
```

```
# 1. Display the first 5 rows of the dataset
print(data.tail())
```

```
# 2. Display the last 5 rows of the dataset
print(data.shape)
```

```
# 3. Get the shape of the dataset
print(data.info())
```

```
# 4. Get a summary of the dataset (info)
print(data.describe())
```

```
# 5. Get basic statistics of the dataset
print(data.isnull().sum())
```

```
# 6. Check for missing value
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace = True)
```

```
# 7. Fill missing values in the 'Age' column with the median age
mode_embarked = data['Embarked'].mode() [0]
data['Embarked'].fillna(mode_embarked, inplace = True)
```

```
# 8. Fill missing values in the 'Embarked' column with the mode
data.drop('Cabin', axis=1, inplace = True)
```

```
# 9. Drop the 'Cabin' column due to many missing values
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

Output:

The screenshot displays a Jupyter Notebook environment with a dark theme. The left pane shows the notebook content, and the right pane shows the output of the code cells.

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. Brown	male	21.01	1	0	2101	5.12	5	Q
2	1	1	Mr. Bradley (Colonel Briggs)	male	30.00	1	0	PC 17009	71.2833	1	C
3	1	3	Miss. Gentry	female	19.00	0	0	3101	5.12	5	Q
4	1	3	Miss. Gentry	female	19.00	0	0	3101	5.12	5	Q
5	1	3	Miss. Gentry	female	19.00	0	0	3101	5.12	5	Q

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mr. Brown, male, 21.01, 1, 0, 2101, 5.12, 5, Q
2, 1, 1, Mr. Bradley (Colonel Briggs), male, 30.00, 1, 0, PC 17009, 71.2833, 1, C
3, 1, 3, Miss. Gentry, female, 19.00, 0, 0, 3101, 5.12, 5, Q
4, 1, 3, Miss. Gentry, female, 19.00, 0, 0, 3101, 5.12, 5, Q
5, 1, 3, Miss. Gentry, female, 19.00, 0, 0, 3101, 5.12, 5, Q
```

The right pane shows the output of the code cells. It includes a summary of the dataset, a list of missing values, and the results of the data cleaning steps. The output is displayed in a table format, showing the expected and actual output for each step.

Test case 1: 1 out of 1 shown test case(s) passed

Expected output: [5 rows x 12 columns]

Actual output: [5 rows x 12 columns]

Terminal: Test cases

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)
```

```
# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)
```

```
# 5. Get the median fare of passengers
```

```
median_fare = data['Fare'].median()
print(median_fare)
```

```
# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

```
# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

```
# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 9. Calculate the survival rate
survival_rate = data['Survived'].mean()
print(survival_rate)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

The screenshot displays a Jupyter Notebook environment with a dark theme. On the left, the notebook contains a title '4.2.6. Titanic Dataset Analysis and Data Cleaning - 2', a description of the task, a list of 10 questions, a table of the dataset's columns, and sample data. The main area shows Python code for loading the dataset, creating a 'FamilySize' column, and calculating various statistics. The right sidebar shows the 'TitanicDataset' output, including execution time, test case results, and a comparison of expected and actual outputs for several variables.

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mrs. Brown	male	29.0	1	0	12345	53.1	C	S
2	1	1	Mr. Smith	male	35.0	0	0	67890	51.0	B	C
3	0	3	Mr. Jones	male	19.0	0	0	11223	7.25	D	S
4	1	1	Mr. White	male	58.0	0	0	45678	51.0	A	C
5	0	3	Mr. Green	male	14.0	1	0	98765	15.5	E	S
6	1	1	Mr. Black	male	45.0	0	0	23456	51.0	B	C
7	0	3	Mr. Grey	male	17.0	0	0	34567	7.125	D	S
8	1	1	Mr. Blue	male	55.0	0	0	56789	51.0	A	C
9	0	3	Mr. Yellow	male	16.0	1	0	67890	15.5	E	S
10	1	1	Mr. Purple	male	42.0	0	0	78901	51.0	B	C

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mrs. Brown, male, 29.0, 1, 0, 12345, 53.1, C, S
2, 1, 1, Mr. Smith, male, 35.0, 0, 0, 67890, 51.0, B, C
3, 0, 3, Mr. Jones, male, 19.0, 0, 0, 11223, 7.25, D, S
4, 1, 1, Mr. White, male, 58.0, 0, 0, 45678, 51.0, A, C
5, 0, 3, Mr. Green, male, 14.0, 1, 0, 98765, 15.5, E, S
6, 1, 1, Mr. Black, male, 45.0, 0, 0, 23456, 51.0, B, C
7, 0, 3, Mr. Grey, male, 17.0, 0, 0, 34567, 7.125, D, S
8, 1, 1, Mr. Blue, male, 55.0, 0, 0, 56789, 51.0, A, C
9, 0, 3, Mr. Yellow, male, 16.0, 1, 0, 67890, 15.5, E, S
10, 1, 1, Mr. Purple, male, 42.0, 0, 0, 78901, 51.0, B, C
```

TitanicDataset

Execution time: 0.000 s, 55.00 ms. Maximum time: 0.000 s, 55.00 ms. 1 out of 1 shown test case(s) passed.

Test case 1: **Expected output** vs **Actual output**

Expected output	Actual output
29.49911764795882	29.49911764795882
14.4542	14.4542
1 ... 491	1 ... 491
1 ... 216	1 ... 216
2 ... 184	2 ... 184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0 ... 577	0 ... 577
1 ... 314	1 ... 314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0 ... 549	0 ... 549
1 ... 342	1 ... 342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0 ... 189908	0 ... 189908
1 ... 742038	1 ... 742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S). 3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
```

```
print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```



```
print(data.groupby('Pclass')['Fare'].mean())
```

```
# 6. Get the average age by class
print(data.groupby('Pclass')['Age'].mean())
```

```
# 7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())
```

```
# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())
```

```
# 9. Get the number of survivors by class
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

```
# 10. Get the number of non-survivors by class
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

The screenshot displays a Jupyter Notebook interface with a dark theme. The left pane shows a notebook titled "4.2.7. Titanic Dataset Analysis and Data Cleaning - 3". It contains a list of 10 tasks for analysis, a table of the dataset's columns, and a sample of the data. The right pane shows the execution of the code, with a green status bar indicating "1 out of 1 shown test case(s) passed". Below this, the "Expected output" and "Actual output" for the first test case are shown, displaying the results of the groupby operations for Pclass, Survived, and FamilySize.

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
2	0	1	Mr. Bradley, Mr. William	male	29	0	0	PC 17599	51.0	C85	C
3	1	3	Ms. Gonsky, Mrs. John	female	36	1	0	PC 17599	51.0	C85	C
4	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
5	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
6	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
7	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
8	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
9	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S
10	1	3	Mr. Brown, Mr. James	male	22	0	0	310127	53.1	5	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, ParCh, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
2, 0, 1, Mr. Bradley, Mr. William, male, 29, 0, 0, PC 17599, 51.0, C85, C
3, 1, 3, Ms. Gonsky, Mrs. John, female, 36, 1, 0, PC 17599, 51.0, C85, C
4, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
5, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
6, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
7, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
8, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
9, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
10, 1, 3, Mr. Brown, Mr. James, male, 22, 0, 0, 310127, 53.1, 5, S
```

Execution results:

Average time: 0.079 s, Maximum time: 0.079 s, 1 out of 1 shown test case(s) passed

Test case 1: Expected output vs Actual output

Expected output	Actual output
Pclass:	Pclass:
1 ... 0.429638	1 ... 0.429638
2 ... 0.472826	2 ... 0.472826
3 ... 0.242363	3 ... 0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S:	Embarked_S:
0 ... 0.586873	0 ... 0.586873
1 ... 0.336957	1 ... 0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize:	FamilySize:
0 ... 0.387538	0 ... 0.387538
1 ... 0.552795	1 ... 0.552795
2 ... 0.578431	2 ... 0.578431
3 ... 0.724138	3 ... 0.724138
4 ... 0.200000	4 ... 0.200000
5 ... 0.136364	5 ... 0.136364
6 ... 0.333333	6 ... 0.333333
7 ... 0.400000	7 ... 0.400000
8 ... 0.400000	8 ... 0.400000
9 ... 0.400000	9 ... 0.400000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone:	IsAlone:
0 ... 0.585658	0 ... 0.585658
1 ... 0.387538	1 ... 0.387538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass:	Pclass:

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
```

```
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
```

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
```

```
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
```

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
```

```
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
```

```
non_survivors_by_embarked_s = data[data['Survived'] ==
```

```
0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

```
# 5. Calculate the percentage of children (Age < 18) who survived
```

```
children = data[data['Age'] < 18]
```

```
children_survival_rate = children['Survived'].mean()
```

```
print(children_survival_rate)
```

```
# 6. Calculate the percentage of adults (Age >= 18) who survived
```

```
adults = data[data['Age'] >= 18]
```

```
adults_survival_rate = adults['Survived'].mean()
```

```
print(adults_survival_rate)
```

```
# 7. Get the median age of survivors
```

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
```

```
print(median_age_survivors)
```

```
# 8. Get the median age of non-survivors
```

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
```

```
print(median_age_non_survivors)
```

```
# 9. Get the median fare of survivors
```

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
```

```
print(median_fare_survivors)
```

```
# 10. Get the median fare of non-survivors
```

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
```

```
print(median_fare_non_survivors)
```

Output:

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. James Watson	male	21.0	0	0	520962349	5.44		S
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	19.0	0	0	175996620	21.01	C85	C
3	1	3	Mr. John Law (John Law)	female	19.0	0	0	108510010	7.25		S
4	1	1	Mr. Joseph North (Miss Mary)	female	19.0	0	0	110800010	11.33	C123	S
5	1	1	Mr. William North (Miss Mary)	female	19.0	0	0	110800010	11.33		S
6	1	3	Mr. James Watson	male	21.0	0	0	520962349	5.44		S
7	1	3	Mr. James Watson	male	21.0	0	0	520962349	5.44		S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. James Watson, male, 21.0, 0, 0, 520962349, 5.44, , S
2, 1, 1, Mr. John Bradley (Florence Briggs Thayer), female, 19.0, 0, 0, 175996620, 21.01, C85, C
3, 1, 3, Mr. John Law (John Law), female, 19.0, 0, 0, 108510010, 7.25, , S
4, 1, 1, Mr. Joseph North (Miss Mary), female, 19.0, 0, 0, 110800010, 11.33, C123, S
5, 1, 1, Mr. William North (Miss Mary), female, 19.0, 0, 0, 110800010, 11.33, , S
6, 1, 3, Mr. James Watson, male, 21.0, 0, 0, 520962349, 5.44, , S
7, 1, 3, Mr. James Watson, male, 21.0, 0, 0, 520962349, 5.44, , S
```

TitanicDataset

1 import pandas as pd

2 import numpy as np

3

4 # Load the Titanic dataset

Average time: 0.048 s

Maximum time: 48.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Female: 233

Female: 233

male: 109

male: 109

Name: Sex, dtype: int64

Name: Sex, dtype: int64

male: 408

male: 408

Female: 81

Female: 81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

1: 217

1: 217

0: 125

0: 125

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

1: 427

1: 427

0: 122

0: 122

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

0.539823888495575

0.539823888495575

0.581815119767905

0.581815119767905

28.0

28.0

28.0

28.0

28.0

28.0